06-03 에러 처리 및 복구 분석

개요

Rain BitTorrent 클라이언트는 네트워크 환경의 불안정성과 다양한 오류 상황에 대응하기 위해 포괄적인 에러 처리 및 복구 메커니즘을 구현합니다. 이 문서는 Rain에서 사용하는 네트워크 오류 처리 패턴, 재시작 데이터 관리, 손상된 데이터 복구, 그리고 그레이스풀 셧다운 기법을 상세히 분석합니다.

에러 타입 분류 및 처리 전략

커스텀 에러 타입

Rain은 에러의 성격에 따라 구체적인 에러 타입을 정의합니다:

```
// 입력 관련 에러
type InputError struct {
err  error
}

func (e *InputError) Error() string {
return  "input error: " + e.err.Error()
}

func (e *InputError) Unwrap() error {
return  e.err
```

```go
}


// 트래커 announce 에러
type AnnounceError struct {
err *announcer.AnnounceError
}


func (e *AnnounceError) Error() string {
return  e.err.Message
}


func (e *AnnounceError) Unknown() bool {
return  e.err.Unknown  // 예상치 못한 에러인지 판단
}
```

에러 분류 체계

Rain은 에러를 다음과 같이 분류하여 처리합니다:

1. **예상 가능한 에러**: 네트워크 오류, DNS 오류, 트래커 오류
2. **예상치 못한 에러**: 시스템 오류, 프로그래밍 오류
3. **복구 가능한 에러**: 일시적 네트워크 문제, 파일 락 등
4. **치명적 에러**: 디스크 공간 부족, 권한 문제 등

네트워크 오류 처리 패턴

피어 연결 오류 처리

피어 연결에서 발생하는 다양한 오류를 체계적으로 처리합니다:

```
func (t *torrent) closePeer(pe *peer.Peer) {
if  pe.Closed {
return  // 이미 닫힌 피어는 무시
}
pe.Close()
pe.Closed = true
// 진행 중인 다운로드 정리
if  pd, ok := t.pieceDownloaders[pe];  ok {
t.closePieceDownloader(pd)
}
// 메타데이터 다운로드 정리
if  id, ok := t.infoDownloaders[pe];  ok {
t.closeInfoDownloader(id)
}
// 피어 관련 데이터 정리
delete(t.peers, pe)
delete(t.incomingPeers, pe)
delete(t.outgoingPeers, pe)
delete(t.peerIDs, pe.ID)
delete(t.connectedPeerIPs, pe.IP())
```

```
// 피스 피커와 언초커에 연결 해제 알림
if  t.piecePicker != nil {
t.piecePicker.HandleDisconnect(pe)
}
t.unchoker.HandleDisconnect(pe)
// PEX에서 피어 제거
t.pexDropPeer(pe.Addr())
// 새로운 연결 시도
t.dialAddresses()
// 메트릭 업데이트
t.session.metrics.Peers.Dec(1)
}
```

연결 재시도 메커니즘

네트워크 오류 시 지능적인 재시도 로직을 구현합니다:

```
func (t *torrent) dialAddresses() {
if  t.completed {
return
}
peersConnected := func() int {
return  len(t.outgoingPeers) + len(t.outgoingHandshakers)
}
```

```
// 최대 연결 수까지 새로운 연결 시도
for peersConnected() < t.session.config.MaxPeerDial {
addr, src := t.addrList.Pop()
if addr == nil {
t.setNeedMorePeers(true) // 더 많은 피어 요청
return
}
ip := addr.IP.String()
if _, ok := t.connectedPeerIPs[ip]; ok {
continue // 이미 연결된 IP는 스킵
}
h := outgoinghandshaker.New(addr, src)
t.outgoingHandshakers[h] = struct{}{}
t.connectedPeerIPs[ip] = struct{}{}
// 비동기 핸드셰이크 시작
go h.Run(
t.session.config.PeerConnectTimeout,
t.session.config.PeerHandshakeTimeout,
t.peerID,
t.infoHash,
t.outgoingHandshakerResultC,
t.session.extensions,
t.session.config.DisableOutgoingEncryption,
```

```
t.session.config.ForceOutgoingEncryption,
)
}
}
```

타임아웃 기반 오류 감지

다양한 타임아웃을 설정하여 네트워크 문제를 조기에 감지합니다:

```
func (t *torrent) resolveAndAddPeer(host  string, port  int) {
ctx, cancel := context.WithCancel(context.Background())
done := make(chan  struct{})
go  func() {
select {
case <-t.closeC: // 토렌트 종료
case <-done: // 정상 완료
}
cancel() // 컨텍스트 취소
}()
// DNS 해석에 타임아웃 적용
ip, err := resolver.ResolveIPv4(ctx, t.session.config.DNSResolveTimeout, host)
if  err != nil {
return  // 타임아웃 또는 DNS 오류 시 조용히 실패
}
```

```
addrs := []*net.TCPAddr{{IP: ip, Port: port}}

t.handleNewPeers(addrs, peersource.Manual)

}
```

재시작 데이터 관리 (`resumer` 패키지)

BoltDB 기반 지속성

Rain은 BoltDB를 사용하여 토렌트 상태를 안전하게 저장합니다:

```
type Resumer struct {

db *bbolt.DB

bucket []byte

}


func (r *Resumer) Write(torrentID  string, spec *Spec) error {

return  r.db.Update(func(tx *bbolt.Tx) error {

b, err := tx.Bucket(r.bucket).CreateBucketIfNotExists([]byte(torrentID))

if  err != nil {

return  err

}

// 모든 필드를 원자적으로 저장

_ = b.Put(Keys.InfoHash, spec.InfoHash)

_ = b.Put(Keys.Port, []byte(strconv.Itoa(spec.Port)))
```

```
_ = b.Put(Keys.Name, []byte(spec.Name))

_ = b.Put(Keys.Bitfield, spec.Bitfield)

_ = b.Put(Keys.BytesDownloaded, []byte(strconv.FormatInt(spec.BytesDownloaded, 1

_ = b.Put(Keys.BytesUploaded, []byte(strconv.FormatInt(spec.BytesUploaded, 10)))

_ = b.Put(Keys.Started, []byte(strconv.FormatBool(spec.Started)))

// ... 기타 필드들

return nil

})

}
```

버전 관리 및 마이그레이션

재시작 데이터의 호환성을 위해 버전 관리를 구현합니다:

```
const LatestVersion = 3


func (s *Session) parseInfo(b []byte, version int) (*metainfo.Info, error) {

var useUTF8Keys bool

var hidePaddings bool

switch version {

case 1:

// 기본 설정

case 2:

useUTF8Keys = true
```

```
case 3:
useUTF8Keys = true
hidePaddings = true
default:
return nil, fmt.Errorf("unknown resume data version: %d", version)
}
i, err := metainfo.NewInfo(b, useUTF8Keys, hidePaddings)
if err != nil {
return nil, err
}
if i.NumPieces > s.config.MaxPieces {
return nil, fmt.Errorf("too many pieces: %d", i.NumPieces)
}
return i, nil
}
```

데이터 마이그레이션 처리

구 버전 데이터를 새 형식으로 자동 마이그레이션합니다:

```
func (r *Resumer) Read(torrentID string) (spec *Spec, err error) {
// 패닉 복구 메커니즘
defer debug.SetPanicOnFault(debug.SetPanicOnFault(true))
defer func() {
```

```go
if r := recover(); r != nil {
err = fmt.Errorf("cannot read torrent %q from db: %s", torrentID, r)
}
}()
err = r.db.Update(func(tx *bbolt.Tx) error {
// ... 데이터 읽기 로직
// 트래커 데이터 마이그레이션 예시
value = b.Get(Keys.Trackers)
if value != nil {
err = json.Unmarshal(value, &spec.Trackers)
if err != nil {
// 구 형식 []string 시도
trackers := make([]string, 0)
err = json.Unmarshal(value, &trackers)
if err != nil {
return err
}
// 새 형식 [][]string으로 변환
spec.Trackers = make([][]string, len(trackers))
for i, t := range trackers {
spec.Trackers[i] = []string{t}
}
// 새 형식으로 저장
```

```
bt, err := json.Marshal(spec.Trackers)
if err != nil {
return err
}
err = b.Put(Keys.Trackers, bt)
if err != nil {
return err
}
}
}
return nil
})
return
}
```

부분 업데이트 최적화

성능을 위해 필요한 필드만 업데이트하는 메서드를 제공합니다:

```
func (r *Resumer) WriteBitfield(torrentID string, value []byte) error {
return r.db.Update(func(tx *bbolt.Tx) error {
b := tx.Bucket(r.bucket).Bucket([]byte(torrentID))
if b == nil {
return nil // 토렌트가 없으면 조용히 무시
```

```
}
return b.Put(Keys.Bitfield, value)
})
}

func (r *Resumer) WriteStarted(torrentID string, value bool) error {
return r.db.Update(func(tx *bbolt.Tx) error {
b := tx.Bucket(r.bucket).Bucket([]byte(torrentID))
if b == nil {
return nil
}
return b.Put(Keys.Started, []byte(strconv.FormatBool(value)))
})
}
```

손상된 데이터 복구

파일 검증 시스템

Rain은 포괄적인 파일 검증 시스템을 구현합니다:

```
type Verifier struct {
Bitfield *bitfield.Bitfield
Error error
```

```go
closeC chan struct{}
doneC chan struct{}
}

func (v *Verifier) Run(pieces []piece.Piece, progressC chan Progress,
resultC chan *Verifier) {
defer close(v.doneC)
defer func() {
select {
case resultC <- v:
case <-v.closeC:
}
}()

v.Bitfield = bitfield.New(uint32(len(pieces)))
buf := make([]byte, pieces[0].Length)
hash := sha1.New()
for _, p := range pieces {
buf = buf[:p.Length]
_, v.Error = p.Data.ReadAt(buf, 0)
if v.Error != nil {
return // I/O 오류 시 즉시 중단
}
```

```
ok := p.VerifyHash(buf, hash)
if ok {
v.Bitfield.Set(p.Index)
}
// 진행률 보고
select {
case progressC <- Progress{Checked: p.Index + 1}:
case <-v.closeC:
return
}
hash.Reset()
}
}
```

검증 결과 처리

검증 완료 후 손상된 피스를 재다운로드하도록 처리합니다:

```
func (t *torrent) handleVerificationDone(ve *verifier.Verifier) {
if t.verifier != ve {
t.crash("invalid verifier") // 예상치 못한 상황
}
t.verifier = nil
```

```go
if ve.Error != nil {
t.stop(fmt.Errorf("file verification error: %s", ve.Error))
return
}

// 검증된 비트필드 적용
t.mBitfield.Lock()
t.bitfield = ve.Bitfield
t.mBitfield.Unlock()

// 재시작 데이터베이스에 저장
err := t.writeBitfield()
if err != nil {
t.stop(err)
return
}

var haveMessages []peerprotocol.HaveMessage
// 완료된 피스 마킹
for i := uint32(0); i < t.bitfield.Len(); i++ {
if t.bitfield.Test(i) {
t.pieces[i].Done = true
haveMessages = append(haveMessages, peerprotocol.HaveMessage{Index: i})
```

```
}
}

// 누락된 피스가 있으면 다운로드 상태로 변경
if !t.bitfield.All() {
t.completed = false
if  t.completeC == nil {
t.completeC = make(chan  struct{})
}
}

// 피어들에게 보유 피스 알림
for  pe := range  t.peers {
for  _, msg := range  haveMessages {
pe.SendMessage(msg)
}
t.updateInterestedState(pe)
}
}
```

데이터베이스 복구 기능

손상된 데이터베이스를 정리하고 압축하는 기능을 제공합니다:

```go
func (s *Session) CleanDatabase() error {
err := s.db.Update(func(tx *bbolt.Tx) error {
b := tx.Bucket(torrentsBucket)
for _, id := range s.invalidTorrentIDs {
err := b.DeleteBucket([]byte(id))
if err != nil {
return err
}
}
return nil
})
if err != nil {
return err
}
s.invalidTorrentIDs = nil
return nil
}

func (s *Session) CompactDatabase(output string) error {
db, err := bbolt.Open(output, 0600, nil)
if err != nil {
return err
```

```
}
defer db.Close()
// 새 데이터베이스 초기화
err = db.Update(func(tx *bbolt.Tx) error {
_, err2 := tx.CreateBucketIfNotExists(torrentsBucket)
return err2
})
if err != nil {
return err
}
res, err := boltdbresumer.New(db, torrentsBucket)
if err != nil {
return err
}
// 유효한 토렌트만 새 데이터베이스에 복사
for _, t := range s.torrents {
if t.torrent.info == nil {
s.log.Warningf("skipping torrent %s: info is nil", t.torrent.id)
continue
}
spec := &boltdbresumer.Spec{
InfoHash: t.torrent.InfoHash(),
Port: t.torrent.port,
```

```
Name: t.torrent.name,
// ... 기타 필드들
}
err = res.Write(t.torrent.id, spec)
if err != nil {
return err
}
}
return db.Close()
}
```

그레이스풀 셧다운

계층적 종료 처리

Rain은 계층적 구조에 따라 순차적으로 종료를 처리합니다:

```
func (s *Session) Close() error {
close(s.closeC) // 종료 신호 전파

// DHT 먼저 종료
if s.config.DHTEnabled {
s.dht.Stop()
}
```

```go
// 통계 업데이트
s.updateStats()

// 모든 토렌트 병렬 종료
var wg sync.WaitGroup
s.mTorrents.Lock()
wg.Add(len(s.torrents))
for _, t := range s.torrents {
go func(t *Torrent) {
t.torrent.Close() // 각 토렌트 정리
wg.Done()
}(t)
}
wg.Wait() // 모든 토렌트 종료 대기
s.torrents = nil
s.mTorrents.Unlock()

// RPC 서버 종료
if s.rpc != nil {
err := s.rpc.Stop(s.config.RPCShutdownTimeout)
if err != nil {
s.log.Errorln("cannot stop RPC server:", err.Error())
```

```
}
}

// 리소스 정리
s.ram.Close()
s.pieceCache.Close()
s.trackerManager.Close()
s.metrics.Close()
return  s.db.Close()
}
```

토렌트별 정리 작업

각 토렌트는 자체적인 정리 작업을 수행합니다:

```
func (t *torrent) close() {
// 실행 중이면 먼저 중지
t.stop(errClosed)

// 중지 이벤트 announcer 정리
if  t.stoppedEventAnnouncer != nil {
t.stoppedEventAnnouncer.Close()
}
```

```go
// 메트릭 정리
t.downloadSpeed.Stop()
t.uploadSpeed.Stop()
}

var errClosed = errors.New("torrent is closed")
```

리소스 해제 보장

모든 고루틴과 리소스가 적절히 해제되도록 보장합니다:

```go
func (p *Peer) Close() {
p.snubTimer.Stop() // 타이머 정지
if p.PEX != nil {
p.PEX.close() // PEX 고루틴 정리
}
close(p.closeC) // 종료 신호
p.Conn.Close() // 네트워크 연결 종료
p.downloadSpeed.Stop() // 메트릭 정리
p.uploadSpeed.Stop()
<-p.doneC // 고루틴 종료 대기
}

func (p *Peer) Run(messages chan Message, pieces chan PieceMessage,
```

```
snubbed, disconnect  chan *Peer) {

defer  close(p.doneC) // 함수 종료 시 완료 신호

go  p.Conn.Run()

for {

select {

// ... 다양한 메시지 처리

case <-p.closeC:

return  // 종료 신호 시 즉시 반환

}

}

}
```

오류 복구 전략

자동 복구 메커니즘

Rain은 다양한 자동 복구 메커니즘을 구현합니다:

1. **피어 연결 복구**: 연결이 끊어진 피어를 자동으로 교체
2. **트래커 복구**: 실패한 트래커를 다른 트래커로 대체
3. **데이터 복구**: 손상된 피스를 자동으로 재다운로드
4. **메타데이터 복구**: 손상된 토렌트 정보를 재다운로드

백오프 전략

반복적인 실패에 대해 지수 백오프를 적용합니다:

```go
// 트래커 announce 실패 시 재시도 간격 증가
func (a *Announcer) calculateNextAnnounce(err  error) time.Duration {
if  err != nil {
// 실패 시 재시도 간격을 두 배로 증가 (최대 1시간)
a.backoffDuration = min(a.backoffDuration*2, time.Hour)
return  a.backoffDuration
}
// 성공 시 정상 간격으로 복귀
a.backoffDuration = a.config.MinAnnounceInterval
return  a.interval
}
```

상태 일관성 보장

시스템 상태의 일관성을 보장하기 위한 검증 로직을 구현합니다:

```go
func (t *torrent) crash(msg  string) {
t.log.Errorf("torrent crashed: %s", msg)
t.stop(fmt.Errorf("torrent crashed: %s", msg))
// 디버그 정보 수집
debug.PrintStack()
// 상태 정리
t.reset()
}
```

```go
func (t *torrent) reset() {
// 모든 연결 정리
for pe := range t.peers {
t.closePeer(pe)
}
// 다운로더 정리
for _, pd := range t.pieceDownloaders {
t.closePieceDownloader(pd)
}
// 상태 초기화
t.pieces = nil
t.bitfield = nil
t.piecePicker = nil
}
```

결론

Rain BitTorrent 클라이언트의 에러 처리 및 복구 시스템은 다음과 같은 핵심 원칙을 따릅니다:

1. **계층적 에러 처리**: 에러 타입에 따른 적절한 처리 전략
2. **자동 복구**: 일시적 문제에 대한 자동 복구 메커니즘
3. **데이터 무결성**: 지속적인 검증과 복구를 통한 데이터 보호
4. **그레이스풀 셧다운**: 안전한 종료를 통한 데이터 손실 방지

5. **상태 일관성**: 시스템 상태의 일관성 보장

이러한 메커니즘들은 불안정한 네트워크 환경에서도 안정적인 BitTorrent 클라이언트 운영을 가능하게 하며, 다른 분산 시스템 개발에도 적용할 수 있는 검증된 패턴들입니다.

관련 문서

선행 지식

- 01-system-architecture.md - 전체 시스템 아키텍처와 컴포넌트 관계
- 03-01-torrent-lifecycle-process.md - 토렌트 라이프사이클과 상태 관리

관련 주제

- 06-02-concurrency-analysis.md - 동시성 환경에서의 에러 처리
- 04-01-networking-packages-analysis.md - 네트워크 오류 처리

후속 학습

- 07-01-library-usage-guide.md - 에러 처리 API 사용법
- 06-01-performance-optimization-analysis.md - 복구 성능 최적화

> **네비게이션:** 이전: 동시성 처리 분석 | 목차 | 다음: 라이브러리 사용법 가이드